UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



MANUAL TÉCNICO

Sistema digital para la administración del servicio de agua potable de la Aldea Panyebar

Documento complementario del trabajo de graduación:
Desarrollo de un sistema digital para la administración y control de pagos de servicio de agua potable mediante identificación única con código QR y plataforma web móvil para la cobranza domiciliaria en la Aldea Panyebar, San Juan La Laguna, Sololá.

Elaborado por: Juan Pablo Ixcamparic Escún
Guatemala, 2026

CONTROL DEL DOCUMENTO

Documento	Manual Técnico
Sistema	Sistema digital para la administración del servicio de agua potable
Comunidad	Aldea Panyebar
Año	2026
Autor	Juan Pablo Ixcamparic Escún
Baseline certificado	`fe5dbf9` — `feat(qr): include current responsible obligations` (cierre de la Fase 5)
Referencia Fase 6	`7fda3ad` — `docs(phase6): clarify Azure BACPAC usage`

Este documento constituye el manual técnico del sistema y puede utilizarse de forma independiente. También puede incorporarse como anexo del trabajo de graduación que documenta el desarrollo del sistema, con su propia portada y numeración.

> **NOTA — ALCANCE Y CONFIDENCIALIDAD**
> Este manual describe el estado implementado y las configuraciones observables en el repositorio; no implica que exista un despliegue en producción. No contiene secretos, claves JWT ni cadenas de conexión sensibles.

ÍNDICE

1. Identificación del documento 1
2. Propósito del manual 1
3. Alcance técnico del sistema 1
4. Arquitectura general 2
4.1 Backend 2
4.2 Frontend 2
5. Stack tecnológico 2
6. Estructura general del repositorio 3
7. Entornos y configuración general 4
7.1 Entorno de desarrollo 4
7.2 Entorno de entrega 4
8. Configuración del backend 4
9. Base de datos y persistencia 6
10. Migraciones de Entity Framework Core 6
10.1 Preparación controlada de una base nueva 7
10.2 Aprovisionamiento del primer administrador 8
11. Seguridad, autenticación y autorización 8
11.1 Autenticación 8
11.2 Contraseñas 8
11.3 Autorización basada en permisos 9
11.4 Usuarios, roles y permisos 9
11.5 Auditoría 9
11.6 Buenas prácticas operativas 10
12. Identificación de suministros mediante NIS y código QR 10
12.1 NIS 10
12.2 Token técnico y código QR 10
12.3 Consulta pública mediante QR 10
12.4 Cambio de responsable 11
13. Cuotas y obligaciones 11
13.1 Cuotas 11
13.2 Obligaciones 11
14. Jornadas comunitarias 12
15. Pagos y comprobantes 12
15.1 Registro y aplicación de pagos 12
15.2 Comprobantes 13
15.3 Anulación y conservación histórica 13
16. Gestión financiera 13
17. Dashboard y reportes 14
17.1 Dashboard 14
17.2 Reportes 14
18. Programación del abastecimiento 14
19. Procesos administrativos del suministro 15
19.1 Solicitud de nuevo servicio 15
19.2 Cancelación, reconexión e historial del suministro 15
20. Administración del Comité 16
20.1 Integrantes y cargos 16

21. Compilación y ejecución 16
21.1 Prerrequisitos 16
21.2 Restauración y compilación del backend 16
21.3 Pruebas automatizadas 17
21.4 Frontend 17
21.5 Ejecución local 17
21.6 Variables y secretos 18
22. Mantenimiento y actualización 18
23. Diagnóstico y solución de problemas 19
24. Condiciones para una futura implementación operativa 20
24.1 Arquitectura de referencia 20
24.2 Requisitos del alojamiento 21
24.3 Condición futura de RNF-11 21
25. Requisitos de respaldo y recuperación 22
25.1 Respaldo automático 22
25.2 Respaldo manual 22
25.3 Restauración controlada 22
25.4 Prueba futura de RNF-10 23
25.5 Seguridad de los respaldos 23
26. Alternativas de alojamiento para una futura implementación 23
26.1 Alternativa A: SmarterASP.NET 23
26.2 Backups de SmarterASP.NET 24
26.3 Alternativa B: Microsoft Azure 24
26.4 Backups de Azure SQL 24
26.5 Portabilidad y archivado mediante BACPAC en Azure SQL 24
26.6 Tabla comparativa 25
26.7 Criterios para una decisión futura 25
26.8 Fuentes de referencia consultadas 26

1. IDENTIFICACIÓN DEL DOCUMENTO

Este Manual Técnico documenta la versión desarrollada y preparada durante la Fase 6 del trabajo de graduación para una futura implementación operativa.

El baseline funcional certificado es:

`fe5dbf9` — `feat(qr): include current responsible obligations`

Este commit representa el checkpoint funcional certificado de cierre de la Fase 5. Como referencia del estado técnico y documental auditado durante el cierre de la Fase 6 se conserva también:

`7fda3ad` — `docs(phase6): clarify Azure BACPAC usage`

Esta referencia de Fase 6 no sustituye el baseline funcional certificado de la Fase 5 ni altera la historia del repositorio.

2. PROPÓSITO DEL MANUAL

El manual proporciona la información técnica necesaria para comprender la arquitectura, la configuración, el despliegue, el mantenimiento y la recuperación del sistema. Su contenido describe el estado implementado y las configuraciones observables en el repositorio; no implica que exista un despliegue definitivo en producción.

3. ALCANCE TÉCNICO DEL SISTEMA

El sistema implementado es una aplicación web responsiva compuesta por un frontend React y una API/backend ASP.NET Core, con persistencia relacional en SQL Server. La primera versión requiere comunicación con el servidor para operar; no implementa una aplicación móvil nativa ni funcionamiento offline.

El alcance funcional respaldado por el código incluye:

- autenticación y autorización administrativa mediante JWT y permisos;
- gestión administrativa de personas, sectores, suministros y responsables;
- identificación de suministros mediante NIS y código QR;
- cuotas y obligaciones;
- jornadas comunitarias, participantes, asistencia y obligaciones derivadas;
- pagos y aplicaciones de pagos;
- finanzas y egresos;
- dashboard y reportes;
- programación y procesos de abastecimiento;
- administración del Comité;
- auditoría de operaciones administrativas.

Estas capacidades se encuentran representadas por los servicios de aplicación, los controladores, las entidades del dominio y los conjuntos de entidades del contexto de persistencia. El detalle de cada flujo operativo debe consultarse en el código y en sus contratos específicos.

4. ARQUITECTURA GENERAL

4.1 Backend

La solución .NET se organiza en los siguientes proyectos:

- **Panyebar.Api**: aplicación web que configura el host ASP.NET Core, los controladores, Swagger/OpenAPI, CORS, autenticación JWT y autorización.
- **Panyebar.Application**: servicios y contratos de aplicación para los módulos funcionales del sistema.
- **Panyebar.Domain**: entidades y tipos propios del dominio del sistema.
- **Panyebar.Infrastructure**: persistencia con Entity Framework Core y SQL Server, registro de dependencias, seguridad y adaptadores de infraestructura.

La descripción anterior se limita a las responsabilidades observables en los proyectos y en su configuración; no presupone patrones arquitectónicos que no estén demostrados por el repositorio.

El contexto `PanyebarDbContext` expone entidades para personas, sectores, suministros, relaciones de responsables, cuotas, obligaciones, jornadas, participaciones, pagos, aplicaciones de pagos, egresos, usuarios y permisos, administración del Comité, auditoría, abastecimiento, procesos de suministro y solicitudes de nuevo servicio. Las configuraciones de persistencia se aplican desde el ensamblado de infraestructura y la base de datos se configura con el proveedor de SQL Server.

4.2 Frontend

El frontend se organiza mediante React y Vite. Sus directorios principales son:

- `app`: composición y configuración de la aplicación;
- `assets`: recursos estáticos;
- `components`: componentes reutilizables de interfaz;
- `config`: configuración del frontend, incluida la base de la API;
- `pages`: páginas de la aplicación;
- `services`: comunicación con los servicios backend;
- `utils`: utilidades compartidas.

El frontend se comunica con la API mediante solicitudes HTTP. Puede utilizar la variable `VITE_API_BASE_URL`; cuando no se configura, la base utilizada es `/api`. En desarrollo, Vite define un proxy de `/api` hacia `http://localhost:5166`. Esta configuración corresponde al entorno local y no determina necesariamente la configuración que se utilizará en producción.

5. STACK TECNOLÓGICO

Las versiones siguientes se obtienen de `global.json`, los archivos de proyecto del backend, `frontend/package.json` y el proyecto de pruebas.

Área	Tecnología o paquete	Versión o configuración declarada
Backend	.NET	`net8.0`
Backend	ASP.NET Core	Incluido mediante el SDK web y `Microsoft.AspNetCore.App`

Área	Tecnología o paquete	Versión o configuración declarada
Backend	Entity Framework Core	`8.0.0`
Backend	SQL Server	Proveedor `Microsoft.EntityFrameworkCore.SqlServer` `8.0.0`
Backend	JWT Bearer	`Microsoft.AspNetCore.Authentication.JwtBearer` `8.0.0`
Backend	Swashbuckle.AspNetCore	`6.6.2`
Frontend	React	`^19.2.8`
Frontend	React DOM	`^19.2.8`
Frontend	React Router DOM	`^7.18.3`
Frontend	Tailwind CSS	`^4.3.3`
Frontend	Vite	`^8.2.0`
Frontend	qrcode.react	`^4.2.0`
Frontend	Oxlint	`^1.75.0`
Pruebas	xUnit	`2.5.3`
Pruebas	Microsoft.NET.Test.Sdk	`17.8.0`
Pruebas	Microsoft.EntityFrameworkCore.InMemory	`8.0.0`
Pruebas	coverlet.collector	`6.0.0`

El proyecto backend tiene como framework objetivo `net8.0`. Las versiones con prefijo `^` corresponden a rangos declarados por el gestor de paquetes del frontend, no a una afirmación de que el entorno local tenga una versión distinta instalada.

El archivo `global.json` solicita el SDK `8.0.424` y establece `rollForward: latestPatch`; por ello, un patch posterior compatible puede ser aceptado por esa configuración. .NET 10 no forma parte del framework objetivo del sistema.

6. ESTRUCTURA GENERAL DEL REPOSITORIO

La estructura relevante del repositorio es:

```
backend/
  Panyebar.sln
  src/
    Panyebar.Api/
    Panyebar.Application/
    Panyebar.Domain/
    Panyebar.Infrastructure/
  tests/
    Panyebar.Security.Tests/
frontend/
  src/
    app/
    assets/
    components/
    config/
    pages/
    services/
    utils/
docs/
```

```
global.json
```

En `backend/src` se encuentran los cuatro proyectos de la solución y en `backend/tests` el proyecto de pruebas observado. `frontend/src` contiene el código React, sus páginas, componentes, servicios y utilidades. Los directorios generados o de salida `node_modules`, `bin`, `obj` y `dist` no son componentes arquitectónicos del sistema y no forman parte de esta estructura técnica.

7. ENTORNOS Y CONFIGURACIÓN GENERAL

7.1 Entorno de desarrollo

El entorno local observado está compuesto por:

- frontend ejecutado con Vite;
- backend ejecutado con ASP.NET Core;
- SQL Server Express local como proveedor de persistencia;
- `PublicWeb:BaseUrl` configurado para el frontend local;
- proxy de Vite para enrutar `/api` hacia el backend local.

La configuración de desarrollo contiene parámetros locales para estos servicios. Los secretos, claves JWT y cadenas de conexión sensibles no forman parte de este manual.

7.2 Entorno de entrega

La infraestructura operativa definitiva no se contrata ni configura durante esta fase. Su selección y configuración quedan condicionadas a una futura decisión de adopción operativa del Comité y de la comunidad. Actualmente no existe una URL pública productiva, no se afirma que exista producción y no se afirma la existencia de respaldos productivos. Si frontend y API se sirven desde orígenes distintos, `PublicWeb:BaseUrl` debe contener exactamente el origen autorizado, con esquema `http` o `https` y sin ruta. Si se usa el mismo origen, debe dejarse vacío y el frontend debe utilizar `/api`.

Los requisitos RNF-10 y RNF-11 no se presentan como verificados en este documento.

8. CONFIGURACIÓN DEL BACKEND

El backend utiliza el sistema de configuración de ASP.NET Core, que combina los archivos de configuración y las variables del entorno según el entorno de ejecución. La configuración efectiva debe proporcionar los valores requeridos sin incorporar secretos al repositorio.

La entrada `ConnectionStrings:DefaultConnection` es obligatoria. El método `AddInfrastructure` obtiene esta cadena mediante la configuración y, si no existe o está vacía, detiene la inicialización mediante una excepción. Cuando está disponible, se utiliza para configurar `PanyebarDbContext` con el proveedor de SQL Server.

La sección `Jwt` contiene los parámetros de los tokens de acceso:

- `Jwt:Issuer`: emisor esperado del token;

- `Jwt:Audience`: audiencia esperada;
- `Jwt:ExpirationMinutes`: duración de los tokens emitidos, en minutos;
- `Jwt:Key`: clave requerida para firmar y validar los tokens.

La clave JWT no se documenta ni se almacena en este manual. Los secretos deben suministrarse mediante configuración segura del entorno, un almacén de secretos o el mecanismo equivalente disponible para la ejecución; no deben incorporarse al repositorio.

`PublicWeb:BaseUrl` identifica el origen web permitido para el frontend. En el entorno `Development`, `Program.cs` utiliza este valor para configurar el origen permitido por la política CORS `FrontendDevelopment`. Esta estrategia de CORS se conserva para desarrollo. Fuera de `Development`, un valor válido habilita únicamente `ConfiguredFrontend` para ese origen; un valor vacío mantiene el escenario same-origin sin habilitar CORS. No se utiliza `AllowAnyOrigin`.

Los valores locales observables en los archivos de desarrollo y perfiles de ejecución son el frontend en `http://localhost:5173`, el backend HTTP en `http://localhost:5166`, el backend HTTPS en `https://localhost:7164` y una base local de SQL Server Express denominada `PanyebarDb`. Estos valores no representan una configuración de producción.

La equivalencia para variables de entorno de ASP.NET Core es:

```
ConnectionStrings__DefaultConnection=<cadena SQL Server del entorno>
Jwt__Issuer=<emisor no secreto>
Jwt__Audience=<audiencia no secreta>
Jwt__ExpirationMinutes=<duración en minutos>
Jwt__Key=<clave secreta JWT>
PublicWeb__BaseUrl=<origen HTTP/HTTPS autorizado o vacío para same-origin>
ASPNETCORE_ENVIRONMENT=<entorno distinto de Development para operación>
```

`ConnectionStrings__DefaultConnection` y `Jwt__Key` son secretos. Las demás variables no son secretos por sí mismas, aunque deben configurarse de acuerdo con el entorno. Nunca deben incluirse valores reales sensibles en el repositorio. `VITE_API_BASE_URL` se incorpora al bundle del frontend y nunca debe contener secretos; puede omitirse para usar `/api` en el escenario same-origin.

Swagger y Swagger UI se habilitan únicamente cuando `app.Environment.IsDevelopment()` es verdadero. El pipeline también utiliza `UseHttpsRedirection()` para redirigir las solicitudes HTTP según la configuración de ejecución.

Parámetro	Propósito	Requerido	Tratamiento recomendado
`ConnectionStrings:DefaultConnection`	Cadena usada por el contexto para conectarse a SQL Server.	Sí	Suministrarla mediante configuración segura del entorno; no registrar valores sensibles.
`Jwt:Issuer`	Identificar el emisor válido de los tokens.	Sí	Mantenerla consistente entre emisión y validación.
`Jwt:Audience`	Identificar la audiencia válida de los tokens.	Sí	Mantenerla consistente entre emisión y validación.
`Jwt:ExpirationMinutes`	Definir la duración de los tokens emitidos.	Configurado actualmente en `60`	Ajustarla según la política de seguridad del entorno.
`Jwt:Key`	Firmar y validar tokens JWT.	Sí	Protegerla como secreto externo al repositorio y no exponerla en registros.

Parámetro	Propósito	Requerido	Tratamiento recomendado
`PublicWeb:BaseUrl`	Definir el único origen permitido por CORS cuando se requiere.	Sí en `Development`; opcional en same-origin	Usar un origen HTTP/HTTPS sin ruta; dejar vacío si frontend y API comparten origen.
`ASPNETCORE_ENVIRONMENT`	Seleccionar el entorno de ejecución de ASP.NET Core.	Para activar el comportamiento `Development`	Definirlo explícitamente según el entorno; no usar `Development` como configuración productiva.

9. BASE DE DATOS Y PERSISTENCIA

La persistencia utiliza Entity Framework Core 8.0.0 con el proveedor de SQL Server. El contexto principal es `PanyebarDbContext`, ubicado en `backend/src/Panyebar.Infrastructure/Persistence/PanyebarDbContext.cs`.

Sus `DbSet` se agrupan funcionalmente de la siguiente manera:

- **Personas y suministros:** `Personas`, `Sectores`, `Suministros` y `PersonaSuministros`.
- **Cuotas y obligaciones:** `Cuotas`, `Obligaciones`.
- **Jornadas:** `Jornadas`, `ParticipacionesJornada` y `ObligacionesJornada`.
- **Pagos:** `Pagos` y `AplicacionesPago`.
- **Finanzas:** `Egresos`.
- **Seguridad:** `UsuariosAdministrativos`, `Roles`, `Permisos`, `UsuarioRoles` y `RolPermisos`.
- **Administración del Comité:** `AdministracionesComite`, `Cargos` e `IntegrantesAdministracion`.
- **Auditoría:** `Auditorias`.
- **Abastecimiento y procesos:** `ProgramacionesAbastecimiento`, `ProcesosSuministro` y `SolicitudesNuevoServicio`.

En `OnModelCreating`, el contexto define la secuencia `SuministroNisSequence` en el esquema `dbo`, con valor inicial `1` e incremento de `1`. La secuencia aporta el correlativo numérico; no constituye por sí sola el formato completo del NIS. La composición del identificador se realiza en `SuministroNisGenerator` y `NisFormatter`, donde el valor se presenta con el formato `PAN-######` dentro del rango implementado.

El mismo método aplica las configuraciones del ensamblado de Infrastructure mediante `ApplyConfigurationsFromAssembly` y ejecuta `ConfigureUtcDateTimes()`. Las clases de configuración se encuentran en `backend/src/Panyebar.Infrastructure/Persistence/Configurations/` y definen el mapeo de entidades, relaciones, restricciones e índices de persistencia sin concentrar esas reglas en el contexto.

10. MIGRACIONES DE ENTITY FRAMEWORK CORE

Las migraciones de Entity Framework Core se encuentran en `backend/src/Panyebar.Infrastructure/Persistence/Migrations/`. Las migraciones actuales, ordenadas cronológicamente por su identificador, son:

Migración	Finalidad inferible por el nombre
`20260827025307_InitialCreate`	Creación inicial del modelo de datos.
`20260903230736_ConfigureSectorAdministration`	Configuración de la administración de sectores.
`20260904000930_ConfigurePersonAdministration`	Configuración de la administración de personas.
`20260905014010_ConfigureSupplyIdentity`	Configuración de la identidad de los suministros.
`20260905015808_ConfigureSupplyAdministration`	Configuración de la administración de suministros.
`20260908015939_ConfigureSupplyProcesses`	Configuración de los procesos de suministro.
`20260908153046_ConfigureFeesAndObligations`	Configuración de cuotas y obligaciones.
`20260911003707_ConfigureCommunityWorkDays`	Configuración de jornadas de trabajo comunitario.
`20260912004909_ConfigurePayments`	Configuración de pagos.
`20260917033536_ConfigureFinancialManagement`	Configuración de la gestión financiera.
`20260917044659_AllowHistoricalPaymentApplications`	Habilitación de aplicaciones históricas de pagos.
`20260929025905_AddSectorToPersonas`	Incorporación del sector a las personas.
`20261002000100_SeedCommitteeCargoCatalog`	Catálogo inicial de cargos del Comité.

`Program.cs` no evidencia llamadas a `Database.Migrate()`, `Database.MigrateAsync()` ni `EnsureCreated()`. Por tanto, las migraciones no se presentan como aplicadas automáticamente al iniciar la API. Su aplicación debe formar parte de un procedimiento controlado de preparación o actualización de la base de datos. El procedimiento definitivo deberá adaptarse a la infraestructura que se seleccione cuando se autorice la futura implementación operativa; este Manual Técnico ya documenta las condiciones y pasos generales necesarios.

No existe evidencia de un seeding general mediante `HasData` en el código productivo actual. Algunas migraciones históricas sí contienen operaciones SQL idempotentes para incorporar permisos u otros elementos específicos y una de ellas contempla compatibilidad con el usuario existente `demo.admin`. Este usuario se documenta únicamente como referencia de QA o demostración y no como cuenta administrativa definitiva de producción.

La base de desarrollo no debe asumirse como una copia completa del entorno final. Los datos ficticios de prueba y cualquier cuenta usada para demostración no deben presentarse como datos productivos.

10.1 Preparación controlada de una base nueva

Para una futura implementación, una tercera persona autorizada debe crear o proporcionar una base SQL Server vacía y conceder al proceso de migración los permisos necesarios. La cadena no debe escribirse en el repositorio; puede suministrarse mediante User Secrets durante una preparación local controlada o mediante `ConnectionStrings__DefaultConnection` en la configuración segura del entorno. Desde la raíz del repositorio, el procedimiento es:

```
dotnet ef migrations list --project
backend/src/Panyebar.Infrastructure/Panyebar.Infrastructure.csproj --startup-project
backend/src/Panyebar.Api/Panyebar.Api.csproj
dotnet ef database update --project
backend/src/Panyebar.Infrastructure/Panyebar.Infrastructure.csproj --startup-project
backend/src/Panyebar.Api/Panyebar.Api.csproj
dotnet ef migrations list --project
backend/src/Panyebar.Infrastructure/Panyebar.Infrastructure.csproj --startup-project
backend/src/Panyebar.Api/Panyebar.Api.csproj
```

La segunda consulta debe confirmar que no quedan migraciones pendientes. Las migraciones históricas no deben editarse ni reescribirse. La API debe iniciarse solo después de completar esta preparación.

10.2 Aprovisionamiento del primer administrador

El repositorio incluye un comando explícito, no un endpoint público ni un seed automático de cada inicio. Después de aplicar las migraciones y con la API detenida, se define temporalmente el nombre de usuario mediante una variable y se ejecuta:

```
$env:PANYEBAR_BOOTSTRAP_ADMIN_USERNAME = "usuario-administrativo-real"
dotnet run --project backend/src/Panyebar.Api/Panyebar.Api.csproj -- --bootstrap-admin
Remove-Item Env:PANYEBAR_BOOTSTRAP_ADMIN_USERNAME
```

El comando solicita la contraseña sin mostrarla ni registrarla. La contraseña se convierte mediante el hasher existente y solo se almacena el hash. El bootstrap crea un rol técnico inicial con los permisos de consulta y gestión necesarios: `SEGURIDAD.USUARIOS.VER`, `SEGURIDAD.USUARIOS.GESTIONAR`, `SEGURIDAD.ROLES.VER`, `SEGURIDAD.ROLES.GESTIONAR`, `SEGURIDAD.PERMISOS.VER` y `SEGURIDAD.PERMISOS.ASIGNAR`. Rechaza la ejecución si ya existe cualquier usuario administrativo, por lo que una segunda ejecución no duplica ni sobrescribe la administración inicial. Luego, el administrador puede continuar con la administración normal de usuarios, roles y permisos desde la aplicación. No debe usarse `demo.admin` como cuenta productiva ni versionarse una contraseña, un hash productivo o cualquier secreto.

11. SEGURIDAD, AUTENTICACIÓN Y AUTORIZACIÓN

11.1 Autenticación

La API utiliza autenticación JWT Bearer. El flujo observable inicia en `POST /api/auth/login`: se busca un usuario administrativo activo por su nombre de usuario, se verifica la contraseña contra su hash y, si las credenciales son válidas, se emite un token de acceso. La respuesta también obtiene los permisos vigentes del usuario para que el frontend pueda representar las capacidades disponibles.

El token emitido contiene la identidad administrativa en los claims de sujeto, identificador y nombre de usuario. Su emisión utiliza la configuración de `Jwt:Issuer`, `Jwt:Audience` y `Jwt:ExpirationMinutes`. La API valida el issuer, la audience, la vigencia y la clave de firma del token; además, el `ClockSkew` de la validación está configurado en cero. La clave `Jwt:Key` es un secreto de configuración y no se incluye en este documento.

El endpoint `GET /api/auth/me` requiere autenticación y obtiene la identidad a partir de los claims del token para devolver la información del usuario y sus permisos actuales. La duración usada para emitir el token se configura mediante `Jwt:ExpirationMinutes`.

11.2 Contraseñas

Las contraseñas se procesan mediante `PasswordHasher<UsuarioAdministrativo>` a través de `PasswordHasherAdapter`. El sistema guarda el resultado en `UsuarioAdministrativo.PasswordHash` y verifica la contraseña proporcionada contra ese valor; la entidad no está diseñada para almacenar la contraseña en texto legible. La implementación acepta también el resultado de verificación que indica que el hash requiere un nuevo cálculo.

El código impone una longitud mínima de ocho caracteres al crear usuarios administrativos o restablecer contraseñas. No se documenta aquí ningún valor de contraseña ni ninguna credencial concreta.

11.3 Autorización basada en permisos

La autorización se implementa mediante políticas dinámicas. Las políticas con el formato `Permission:<código>` son interpretadas por `PermissionPolicyProvider`, que crea un requisito `PermissionRequirement` y exige primero una identidad autenticada. `PermissionAuthorizationHandler` obtiene el identificador administrativo desde los claims y consulta `IUsuarioPermissionRepository`.

La consulta de permisos comprueba que el usuario esté activo, recorre sus relaciones `UsuarioRol`, considera únicamente los roles activos, recorre sus relaciones `RolPermiso` y finalmente exige que el permiso encontrado esté activo y tenga el código solicitado. La autorización efectiva ocurre cuando esa consulta confirma el permiso.

En los controladores se observan códigos con nomenclatura de módulo y acción, por ejemplo `PERSONAS.VER`, `SUMINISTROS.GESTIONAR` y `SEGURIDAD.AUDITORIA.VER`. Estos ejemplos proceden de los códigos declarados por la aplicación y no constituyen una lista exhaustiva en este manual.

Autenticación, rol, permiso y autorización cumplen funciones distintas:

- **Autenticación:** determina si las credenciales permiten reconocer a un usuario administrativo y emitir un token.
- **Rol:** agrupa una asignación administrativa reutilizable.
- **Permiso:** representa una capacidad técnica identificada por un código.
- **Autorización efectiva:** decide si una solicitud autenticada puede ejecutar una operación, consultando el permiso requerido y el estado vigente del usuario, sus roles y sus permisos.

11.4 Usuarios, roles y permisos

`UsuarioAdministrativo` contiene la identidad administrativa, su hash de contraseña y su estado. `UsuarioRol` relaciona usuarios con roles. `Rol` contiene el nombre, descripción y estado de una agrupación de autorización. `RolPermiso` relaciona cada rol con los permisos que puede conceder, mientras que `Permiso` contiene el código, nombre, descripción y estado de la capacidad técnica.

La asignación efectiva es, por tanto, una relación indirecta: `UsuarioAdministrativo` → `UsuarioRol` → `Rol` → `RolPermiso` → `Permiso`. El código no establece aquí que un rol específico sea obligatorio para producción. Las cuentas usadas para QA o demostración no forman parte de una definición de credenciales productivas.

11.5 Auditoría

Las operaciones administrativas que incorporan auditoría crean registros en la entidad `Auditoria` con el usuario administrativo, la acción, la entidad y su identificador, la fecha y, cuando corresponde, los valores anterior y nuevo. El servicio de administración permite consultar estos registros con filtros por fechas, usuario, acción y entidad, con un límite controlado de resultados.

El código muestra este mecanismo en operaciones administrativas y funcionales concretas, como cambios de responsable, procesos de suministro, gestión de usuarios, cuotas, pagos, finanzas y administración del Comité. No se afirma que absolutamente todas las operaciones del sistema generen un registro de auditoría.

11.6 Buenas prácticas operativas

Las siguientes son recomendaciones operativas y no afirmaciones de que todas ellas estén verificadas o automatizadas por la implementación actual:

- mantener secretos y credenciales fuera del repositorio;
- utilizar credenciales individuales y evitar compartir cuentas administrativas;
- aplicar el principio de mínimo privilegio mediante permisos necesarios;
- cambiar o revocar accesos cuando corresponda;
- utilizar HTTPS en el entorno de entrega.

12. IDENTIFICACIÓN DE SUMINISTROS MEDIANTE NIS Y CÓDIGO QR

12.1 NIS

El NIS pertenece al suministro y se almacena en la propiedad `Suministro.Nis`. Al crear un suministro, `SuministroNisGenerator` solicita el siguiente valor de la secuencia persistente `dbo.SuministroNisSequence`. Después, `NisFormatter` convierte ese correlativo en el formato implementado `PAN-######`; por ejemplo, el primer valor puede representarse conceptualmente como `PAN-000001`.

La secuencia numérica y el formateo son responsabilidades separadas. El modelo configura un índice único sobre `Nis`, por lo que no se admiten dos suministros con el mismo identificador almacenado. El NIS es una identidad del suministro, no de la persona responsable; el cambio de responsable no crea un nuevo NIS. Las operaciones de cambio de responsable conservan el suministro y no asignan nuevamente su NIS.

12.2 Token técnico y código QR

Cada suministro se asocia con `Suministro.CodigoQrToken`. El generador utiliza un valor aleatorio criptográfico de 32 bytes y lo representa como Base64 URL-safe sin el relleno final. El token se almacena asociado al suministro y tiene un índice único.

Al solicitar el QR administrativo, el backend construye una URL absoluta con `PublicWeb:BaseUrl`, la ruta `/suministro/qr/{token}` y el token escapado. En el frontend, `qrcode.react` utiliza ese valor para el renderizado visual mediante `QRCodeSVG`. Por tanto, el token técnico, la URL de consulta y la representación visual son elementos relacionados pero distintos.

El token no contiene directamente datos personales ni financieros y no funciona como contraseña. La consulta QR pública no sustituye la autenticación ni la autorización administrativa: existe una ruta pública específica para la consulta permitida y la consulta administrativa por token permanece protegida por el permiso de suministros.

12.3 Consulta pública mediante QR

La ruta `GET /api/suministros/public/qr/{token}` permite consultar el resumen público asociado con un token válido sin exigir autenticación. La respuesta implementada contiene el NIS, el sector, el estado del suministro, la cantidad de obligaciones pendientes y el total pendiente.

El cálculo considera únicamente obligaciones con estado `Pendiente`: incluye las obligaciones propias del suministro y, cuando existe un responsable vigente, las obligaciones personales pendientes de esa persona. Por ello, las obligaciones pagadas o anuladas no se incluyen, ni se consideran las obligaciones personales de responsables históricos. La respuesta no expone directamente datos personales del responsable ni detalles individuales de las obligaciones.

12.4 Cambio de responsable

El cambio de responsable modifica la relación entre persona y suministro, no la identidad del suministro. Si existía una relación vigente, se registra su fecha de finalización y estado finalizado; después se crea una nueva relación vigente en `PersonaSuministro`. Un índice único filtrado garantiza como máximo un responsable vigente por suministro, mientras que las relaciones anteriores se conservan para el historial.

El suministro conserva su NIS y su `CodigoQrToken` durante este cambio. La consulta pública utiliza el responsable vigente para incorporar sus obligaciones personales pendientes, de modo que no arrastra las obligaciones personales de responsables históricos. La operación también registra la acción, la entidad, el usuario administrativo y los valores anterior y nuevo en `Auditoria`.

13. CUOTAS Y OBLIGACIONES

13.1 Cuotas

`Cuota` representa una definición administrativa de un cobro. Contiene nombre, descripción opcional, monto, periodicidad, fecha inicial y final de vigencia y estado de registro. Las periodicidades implementadas son `Anual` y `Mensual`. El servicio valida el monto con precisión de dos decimales y la coherencia del rango de vigencia.

Una cuota puede utilizarse para generar una obligación para un suministro. En ese proceso se conserva la referencia a `Cuota`, se copia el concepto y monto de la cuota y se establece un período compatible con su periodicidad: `yyyy` para una cuota anual o `yyyy-MM` para una cuota mensual. El sistema impide duplicar una obligación no anulada para la misma cuota, suministro y período. Una modificación posterior de la cuota no reescribe el monto histórico de una obligación ya generada. El código no establece un monto permanente como Q30.

La cuota es la configuración que puede reutilizarse para generar cobros; la obligación es el registro económico concreto, con su propio monto, período, fechas, origen y estado.

13.2 Obligaciones

`Obligacion` representa un importe exigible a un único titular. La persistencia aplica una restricción XOR: cada obligación pertenece a un suministro o a una persona, pero no a ambos ni a ninguno. Por ello, una obligación de cuota se relaciona con un suministro y una obligación personal se relaciona directamente con una persona.

El origen distingue `CuotaOrdinaria`, `Jornada` y `Administrativa`. El registro conserva concepto, monto, período opcional, fecha de generación, vencimiento opcional y estado. Los estados implementados son `Pendiente`, `Pagada` y `Anulada`. La consulta proyecta además `EsMorosa` cuando una obligación pendiente tiene vencimiento y la fecha actual ya lo superó.

La generación administrativa personal recibe varias personas activas, valida que no existan identificadores repetidos y crea una obligación independiente para cada persona seleccionada. Cada registro queda con

`PersonaId`, sin `SuministroId` ni `CuotaId`, y se audita individualmente; no se crea una obligación compartida entre personas.

Las consultas de obligaciones exponen un listado ordenado por fecha de generación e identificador, y una consulta individual. Las operaciones de generación validan referencias, período, montos y duplicidad; la anulación requiere motivo y solo cambia una obligación pendiente a `Anulada`, conservando el registro y su auditoría. Una obligación pagada o anulada no puede anularse nuevamente.

14. JORNADAS COMUNITARIAS

`Jornada` representa una actividad comunitaria dirigida a personas. Contiene nombre, descripción, fecha, horario opcional, ubicación, monto opcional de incumplimiento y estado. Sus estados son `Planificada`, `Cerrada` y `Cancelada`; solo una jornada planificada puede editarse, cancelarse, modificar participantes o cerrarse.

`ParticipacionJornada` relaciona una jornada con una persona y registra el resultado administrativo y una observación opcional. Los resultados observables son `Pendiente`, `Participacion`, `Ausencia` y `AusenciaJustificada`. Una restricción única evita más de una participación para la misma jornada y persona. La jornada no se relaciona aquí con el suministro.

Para cerrar una jornada deben existir participantes y no puede quedar ninguno con resultado `Pendiente`. La ausencia no genera automáticamente una deuda en cualquier circunstancia. Al cerrar, solo las participaciones con resultado `Ausencia` generan obligaciones cuando la jornada tiene configurado un `MontoIncumplimiento` positivo. La participación y la ausencia justificada no generan obligación, y una ausencia sin monto configurado tampoco la genera.

La obligación derivada se asigna a la persona de la participación, tiene origen `Jornada`, queda pendiente, no tiene suministro, cuota, período ni vencimiento, y utiliza como concepto la jornada correspondiente. `ObligacionJornada` relaciona esa obligación con la participación que la originó. Sus índices únicos garantizan como máximo una obligación por participación y una participación por obligación; antes del cierre el servicio también evita volver a enlazar participaciones que ya tengan obligación.

15. PAGOS Y COMPROBANTES

15.1 Registro y aplicación de pagos

Un `Pago` registra una transacción con monto, fecha, concepto, usuario administrativo y estado. Sus obligaciones se relacionan mediante `AplicacionPago`, que conecta cada pago con cada `Obligacion` aplicada:

```
Pago -> AplicacionPago -> Obligacion
```

El registro de pago exige que las obligaciones existan, que sus identificadores no estén repetidos, que todas estén en estado `Pendiente` y que pertenezcan al mismo titular. El titular puede ser una persona o un suministro, pero no se mezclan obligaciones de distintos titulares. El monto recibido debe coincidir exactamente con la suma de los montos de las obligaciones seleccionadas.

En consecuencia, no se admiten pagos parciales, sobrepagos ni saldos a favor. Un mismo pago sí puede aplicar varias obligaciones completas del mismo titular. En una operación válida se crea el pago, se crea una aplicación

por obligación y cada obligación cambia a `Pagada` dentro de la operación transaccional. La combinación `PagoId` y `ObligacionId` es única para evitar duplicar una misma aplicación dentro del mismo pago.

15.2 Comprobantes

El servicio proporciona los datos del comprobante mediante `GetComprobanteAsync`; no crea un archivo PDF ni persiste un documento separado. El número se deriva del identificador del pago con el formato `PAG-######`. El `ComprobantePagoDto` contiene, de forma general, número, identificador, fecha, concepto, total, estado, usuario administrativo, titular y la lista de obligaciones aplicadas con su origen, concepto, período y monto.

La API expone estos datos mediante el endpoint de comprobante y el frontend los utiliza para la representación digital del comprobante. La existencia de este DTO no implica un formato de archivo adicional.

15.3 Anulación y conservación histórica

La anulación solo procede para un pago registrado cuyas aplicaciones existan y cuyas obligaciones continúen pagadas. Cambia el estado del pago a `Anulado`, devuelve las obligaciones asociadas a `Pendiente` y conserva las filas de `Pago` y `AplicacionPago`; por tanto, la relación histórica permanece. La misma obligación puede pagarse nuevamente después de anular el pago anterior, sin eliminar la aplicación histórica.

La operación registra una auditoría con el estado y las obligaciones afectadas. Un pago ya anulado no puede anularse otra vez. Las reglas de validación también impiden registrar pagos sobre obligaciones inexistentes, no pendientes o con montos inconsistentes.

16. GESTIÓN FINANCIERA

La gestión financiera combina pagos registrados y egresos administrativos. No existe una entidad persistente independiente llamada `Ingreso`: los ingresos financieros se derivan de los registros de `Pago` cuyo estado es `Registrado`. Los pagos anulados quedan fuera de los ingresos, movimientos y totales.

`Egreso` es la entidad persistente para gastos administrativos del Comité. Guarda concepto, monto, fecha civil, usuario administrativo y estado. Los estados son `Registrado` y `Anulado`; el monto debe ser positivo y el concepto no puede superar la longitud configurada. Un egreso registrado puede editarse o anularse. La anulación conserva la fila, cambia su estado y excluye el egreso de los movimientos y totales; un egreso anulado no puede editarse ni anularse de nuevo.

El servicio financiero proporciona consultas de ingresos, egresos, movimientos y resumen, con filtros de fecha y, cuando corresponde, estado o tipo de movimiento. El resumen calcula:

```
balance = pagos registrados - egresos registrados
```

Los pagos se filtran usando el día operativo de Guatemala y los egresos usan su fecha civil. Los movimientos distinguen los tipos `Ingreso` y `Egreso`, y el resumen devuelve ingresos totales, egresos totales y balance.

El registro, edición y anulación de egresos generan auditoría con el usuario, la acción, la entidad y los valores anterior y nuevo cuando corresponde. Los pagos registrados y anulados también conservan la trazabilidad mediante las auditorías del servicio de pagos.

Los controladores financieros se protegen con los permisos declarados para finanzas, entre ellos `FINANZAS.VER` y `FINANZAS.GESTIONAR`; estos permisos controlan el acceso a consultas y operaciones administrativas respectivamente.

17. DASHBOARD Y REPORTES

17.1 Dashboard

El dashboard se expone mediante `GET /api/dashboard/resumen` y requiere el permiso `DASHBOARD.VER`. Recibe año y mes, valida que formen un período mensual representable y devuelve un resumen para ese mes administrativo. El cálculo incluye ingresos totales derivados de pagos registrados, egresos totales de egresos registrados, balance, cantidad de pagos registrados, cantidad de obligaciones pendientes y monto de esas obligaciones pendientes.

Los pagos se filtran por su fecha UTC convertida desde el inicio del día operativo de Guatemala. Los egresos se consultan con sus fechas civiles. Las obligaciones pertenecen al mes por `FechaGeneracion`, no por su fecha de vencimiento ni por el texto de su período. El balance del resumen es la diferencia entre ingresos y egresos del intervalo mensual.

El frontend utiliza además la consulta de recaudación por sector para mostrar la distribución territorial de pagos. Esta consulta agrupa aplicaciones de pagos registrados cuyas obligaciones pertenecen a suministros, y suma el monto histórico de esas obligaciones por sector. Las aplicaciones históricas de pagos anulados se conservan, pero no se consideran recaudación.

17.2 Reportes

Los reportes se exponen bajo `GET /api/reportes` y requieren `REPORTES.VER`. Las consultas implementadas son:

- **Pagos:** lista pagos dentro de un rango de fechas, incluyendo su estado, monto, concepto, fecha y usuario administrativo. El reporte conserva tanto registros como anulaciones; solo los registrados representan ingreso.
- **Recaudación por sector:** agrupa por sector las aplicaciones de pagos registrados asociadas con obligaciones de suministros y permite rango de fechas.
- **Obligaciones pendientes:** lista obligaciones pendientes, su titular según corresponda, origen, concepto, monto, período, fechas y condición de mora.
- **Participación en jornadas:** consulta participaciones de personas, resultado, observación, jornada, estado y fecha de la jornada, con rango de fechas civiles.

Los rangos de fechas se validan para impedir que la fecha inicial sea posterior a la final. La implementación no evidencia exportación a Excel o PDF, envío por correo, gráficas adicionales ni analítica predictiva.

18. PROGRAMACIÓN DEL ABASTECIMIENTO

`ProgramacionAbastecimiento` registra la planificación administrativa del abastecimiento para un `Sector`. Contiene fecha, hora inicial, hora final, estado y observación opcional. Los estados implementados son `Programado`, `Completado` y `Cancelado`.

El servicio permite consultar una programación individual o listados filtrados por fecha inicial, fecha final y sector. Los resultados se ordenan por fecha, hora y nombre del sector. Para crear o actualizar una programación se valida que el sector exista y esté activo, que el intervalo horario sea válido y que la observación no supere la longitud permitida. No se admite solapamiento de intervalos programados para el mismo sector y fecha.

También existe una operación de creación recurrente. La recurrencia puede ser semanal, mensual o anual, y se define mediante una cantidad de ocurrencias o una fecha final, pero no ambas. El servicio limita la generación a entre 2 y 52 programaciones y verifica los solapamientos antes de persistir el conjunto.

Una programación en estado `Programado` puede actualizarse, completarse o cancelarse. Las transiciones no ejecutan acciones sobre válvulas, sensores, telemetría, redes hidráulicas ni otros dispositivos: el módulo registra y consulta planificación administrativa por sector. Las operaciones de gestión registran auditoría y requieren `ABASTECIMIENTO.GESTIONAR`; las consultas requieren `ABASTECIMIENTO.VER`.

19. PROCESOS ADMINISTRATIVOS DEL SUMINISTRO

19.1 Solicitud de nuevo servicio

`SolicitudNuevoServicio` representa una solicitud administrativa presentada por una persona para un sector, con dirección de referencia, fecha de solicitud, observación opcional y estado. Sus estados son `Pendiente`, `Aprobada` y `Rechazada`. La creación valida que la persona y el sector estén activos y que la persona no tenga otra solicitud pendiente de nuevo servicio.

Las solicitudes se consultan priorizando las pendientes. Resolver una solicitud solo es posible mientras permanece pendiente y requiere una observación de resolución válida cuando se proporciona. Al aprobarla, el servicio crea el suministro activo, genera su NIS y token QR, lo relaciona con el sector y crea la relación vigente entre el suministro y la persona solicitante. Al rechazarla no se crea un suministro. Por tanto, la solicitud no representa por sí sola un suministro creado automáticamente antes de su aprobación.

La creación y resolución de solicitudes se auditan con el usuario administrativo que ejecuta la operación. La consulta requiere `SUMINISTROS.VER` y la creación, aprobación o rechazo requiere `SUMINISTROS.GESTIONAR`.

19.2 Cancelación, reconexión e historial del suministro

Los procesos operativos administrativos del suministro se representan mediante `ProcesoSuministro`, relacionado con un `Suministro` y con el usuario administrativo que ejecutó la acción. El tipo de proceso puede ser `Cancelacion` o `Reconexion`; cada registro conserva estado anterior, estado nuevo, fecha, motivo obligatorio y observación opcional.

La cancelación solo se aplica a un suministro `Activo` y lo cambia a `Cancelado`. La reconexión solo se aplica a un suministro `Cancelado` y lo devuelve a `Activo`. El motivo tiene longitud validada y la operación registra el proceso histórico junto con una auditoría. El suministro no se elimina, por lo que conserva su registro, NIS, token QR y el historial consultable de procesos.

Las consultas administrativas de procesos requieren `SUMINISTROS.VER`, y las transiciones de cancelación y reconexión requieren `SUMINISTROS.GESTIONAR`. Estas operaciones representan cambios administrativos de estado y no deben interpretarse como eliminación física del suministro ni como automatización del servicio hidráulico.

20. ADMINISTRACIÓN DEL COMITÉ

`AdministracionComite` representa un período de administración con nombre, fecha de inicio, fecha de finalización opcional y estado de registro. Una administración se considera activa cuando su estado es `Activo` y no tiene fecha de finalización.

El servicio permite crear una administración, consultar el listado histórico, obtener una administración y finalizarla. La creación valida nombre, fecha y usuario actor, y no permite otra administración activa al mismo tiempo. La finalización exige una fecha no anterior al inicio, registra `FechaFin` y cambia el estado a `Inactivo`. Las administraciones anteriores se conservan y se devuelven en las consultas históricas.

20.1 Integrantes y cargos

`IntegranteAdministracion` relaciona una administración con una `Persona` y un `Cargo`. `Cargo` es un catálogo con nombre, descripción y estado, y el servicio expone los cargos activos para asignación. La asignación de un integrante exige que la administración esté activa y que la persona y el cargo estén activos.

La asignación evita conservar simultáneamente conflictos de la misma persona o del mismo cargo dentro de una administración: cuando corresponde, reemplaza las asignaciones relacionadas y registra la nueva combinación persona-cargo. Los integrantes se presentan con el nombre de la persona y el nombre del cargo. El modelo no relaciona `IntegranteAdministracion` con `UsuarioAdministrativo`; ser integrante del Comité no equivale a poseer una cuenta de acceso al sistema.

La creación de administraciones, la asignación de integrantes y la finalización requieren `ADMINISTRACION.GESTIONAR`; las consultas y el catálogo de cargos requieren `ADMINISTRACION.VER`. Estas operaciones registran auditoría con el usuario actor, la acción, la entidad y los valores anteriores y nuevos cuando corresponde.

21. COMPILACIÓN Y EJECUCIÓN

21.1 Prerrequisitos

Los prerrequisitos observables del repositorio son:

- un SDK de .NET 8 compatible con `global.json`. El archivo solicita `8.0.424` y permite el avance a un patch compatible mediante `latestPatch`;
- Node.js y npm para restaurar y ejecutar el frontend. El repositorio no fija una versión mínima de Node.js;
- una instancia compatible de SQL Server;
- una cadena `ConnectionStrings:DefaultConnection`, una configuración JWT válida y `PublicWeb:BaseUrl` según el entorno de ejecución.

El proyecto no tiene como framework objetivo .NET 10. La base de datos debe estar preparada mediante el procedimiento controlado de migraciones correspondiente antes de iniciar operaciones que requieran persistencia.

21.2 Restauración y compilación del backend

Desde la raíz del repositorio, la solución backend puede restaurarse y compilarse con:

```
dotnet restore backend/Panyebar.sln
dotnet build backend/Panyebar.sln -c Release
dotnet test backend/tests/Panyebar.Security.Tests/Panyebar.Security.Tests.csproj -c
Release
dotnet publish backend/src/Panyebar.Api/Panyebar.Api.csproj -c Release
```

La solución contiene los proyectos `Panyebar.Api`, `Panyebar.Application`, `Panyebar.Domain` y `Panyebar.Infrastructure`. Estos comandos restauran las dependencias .NET y verifican que los proyectos compilen; no despliegan la API, no crean la base de datos y no aplican migraciones automáticamente.

21.3 Pruebas automatizadas

El proyecto de pruebas se encuentra en `backend/tests/Panyebar.Security.Tests/Panyebar.Security.Tests.csproj`. Puede ejecutarse desde la raíz con:

```
dotnet test backend/tests/Panyebar.Security.Tests/Panyebar.Security.Tests.csproj -c
Release
```

La suite vigente es el criterio operativo: debe finalizar correctamente. En la Fase 5, el baseline certificado registró históricamente `404/404`. Durante la preparación técnica final de la Fase 6 se registraron `417/417` pruebas superadas, con `0` fallidas y `0` omitidas. El aumento corresponde, entre otras pruebas legítimas incorporadas posteriormente, a la cobertura de CORS y del aprovisionamiento controlado del primer administrador. Estas cifras no constituyen un requisito fijo permanente.

21.4 Frontend

Como existe `frontend/package-lock.json`, la restauración reproducible preferente es:

```
cd frontend
npm ci
npm run lint
npm run build
```

Los scripts declarados en `frontend/package.json` son:

```
npm run dev
npm run lint
npm run build
npm run preview
```

`npm run dev` inicia el servidor de desarrollo de Vite. `npm run lint` ejecuta Oxlint y `npm run build` genera el build del frontend mediante Vite. `npm run preview` sirve localmente el resultado construido para revisión; `vite preview` no es el servidor de producción definitivo. `frontend/dist` y la salida generada por `dotnet publish` son artefactos de publicación y no deben versionarse. El hosting futuro debe servir el frontend, ejecutar ASP.NET Core 8 y configurar HTTPS en la infraestructura adoptada.

21.5 Ejecución local

El orden general de preparación local es:

1. disponer de SQL Server y de la base de datos preparada;
2. proporcionar la configuración del backend, sin incluir secretos en el repositorio;
3. iniciar el backend ASP.NET Core;
4. iniciar el frontend Vite.

Los perfiles `Development` observables exponen el backend HTTP en `http://localhost:5166` y HTTPS en `https://localhost:7164`; el frontend local se configura habitualmente en `http://localhost:5173`. Vite utiliza un proxy de `/api` hacia `http://localhost:5166`. Estas URLs corresponden al entorno local, no a un entorno de entrega.

21.6 Variables y secretos

El backend obtiene configuración desde el sistema de configuración de ASP.NET Core. `appsettings.json` contiene parámetros no secretos; la configuración de desarrollo añade la conexión local y `PublicWeb:BaseUrl`. El proyecto API declara un `UserSecretsId`, por lo que User Secrets puede utilizarse durante el desarrollo para valores sensibles sin escribirlos en los archivos versionados.

El frontend documenta sus variables en `frontend/.env.example`. Puede utilizar `VITE_API_BASE_URL` y, si no se define, usa `/api`; también puede definir `VITE_ALLOWED_HOST` para Vite. Las variables con prefijo `VITE_` pueden formar parte del bundle del cliente y no son un almacenamiento seguro de secretos. No deben contener claves JWT, contraseñas ni cadenas de conexión sensibles.

22. MANTENIMIENTO Y ACTUALIZACIÓN

Las actualizaciones deben ejecutarse mediante un procedimiento controlado y adaptado al entorno que se defina para la entrega. Como secuencia general:

1. respaldar la información antes de cambios que puedan afectar datos;
2. identificar el commit o versión actualmente instalada;
3. obtener o preparar la nueva versión;
4. revisar cambios de configuración y variables requeridas;
5. restaurar dependencias .NET y npm usando los archivos declarativos;
6. revisar las migraciones nuevas y su impacto antes de aplicarlas;
7. aplicar las migraciones mediante el procedimiento controlado que corresponda;
8. compilar el backend;
9. ejecutar la suite de pruebas vigente;
10. ejecutar lint y build del frontend;
11. desplegar la versión en el entorno definido;
12. realizar una verificación funcional posterior;
13. conservar una posibilidad de recuperación o rollback conforme a la infraestructura finalmente seleccionada.

Las migraciones existentes forman parte del historial de persistencia y no deben editarse arbitrariamente después de haber sido utilizadas. Los cambios futuros del modelo deben producir nuevas migraciones controladas, que se revisen antes de su aplicación. La API actual no ejecuta migraciones automáticamente al iniciar, por lo que la preparación o actualización de la base de datos debe coordinarse explícitamente.

El rollback de la aplicación y la reversión de la base de datos son operaciones distintas. Recuperar una versión anterior del código no revierte por sí mismo los cambios de esquema ni los datos ya aplicados. El procedimiento de base de datos debe definirse y validarse según la infraestructura real; este manual no incluye comandos destructivos de rollback de producción.

Las actualizaciones de paquetes NuGet y npm deben revisarse y probarse antes de incorporarse. No se recomienda actualizar dependencias de forma indiscriminada ni sin comprobar compatibilidad, compilación, pruebas y comportamiento del frontend.

23. DIAGNÓSTICO Y SOLUCIÓN DE PROBLEMAS

Síntoma	Posible causa	Verificación	Acción recomendada
El backend no inicia y menciona `DefaultConnection`.	Falta o está vacía `ConnectionStrings:DefaultConnection`.	Revisar la configuración efectiva del entorno sin exponer su valor.	Proporcionar la cadena mediante configuración segura y confirmar acceso a SQL Server.
El backend no inicia por JWT.	Falta `Jwt`, `Issuer`, `Audience` o `Key`, o la sección no es válida.	Revisar los nombres de configuración y los mensajes de inicialización.	Completar la configuración segura; no poner la clave en código ni en el repositorio.
Error de conexión a SQL Server.	SQL Server no está disponible, la base no existe o la conexión no es válida.	Confirmar que la instancia esté activa y revisar la configuración local.	Preparar la base mediante el procedimiento controlado y corregir la configuración de conexión.
El frontend no alcanza `/api`.	El backend no está iniciado o el proxy de Vite apunta al destino local esperado.	Revisar el proceso backend y el proxy de `vite.config.js`.	Iniciar ambos servicios y verificar la ruta `/api`; revisar `VITE_API_BASE_URL` si se configuró.
Error CORS durante `Development`.	`PublicWeb:BaseUrl` no coincide con el origen del frontend.	Comparar el origen del navegador con `PublicWeb:BaseUrl`.	Configurar el origen local correcto; no trasladar automáticamente esta estrategia a producción.
Respuesta `401 Unauthorized`.	Falta el token Bearer o no supera la autenticación/validación JWT.	Revisar el login, el encabezado `Authorization`, issuer, audience y vigencia.	Autenticar nuevamente y revisar la configuración JWT sin exponer la clave.
Respuesta `403 Forbidden`.	La identidad está autenticada, pero no tiene el permiso requerido.	Revisar el permiso de la política y las asignaciones activas del usuario.	Asignar solo el permiso necesario mediante el flujo administrativo autorizado.
La API requiere una migración pendiente.	La base no corresponde al modelo o no se aplicó una migración nueva.	Comparar el historial de migraciones con el código y el modelo esperado.	Revisar y aplicar la migración mediante el procedimiento controlado; no editar migraciones usadas arbitrariamente.
Falla la compilación backend.	Error de código, SDK incompatible, dependencia restaurada incorrectamente o configuración de entorno.	Ejecutar `dotnet build backend/Panyebar.sln` y revisar el primer error relevante.	Corregir la causa reportada y volver a compilar; no ocultar el error desactivando validaciones.
Falla lint o build frontend.	Error de sintaxis, lint, dependencia o configuración Vite.	Ejecutar `npm run lint` y `npm run build` desde `frontend/`.	Corregir el diagnóstico y repetir ambos comandos antes de entregar el build.
El QR no encuentra el suministro.	Token inválido, inexistente o URL pública mal formada.	Verificar el token permitido, `PublicWeb:BaseUrl` y la ruta pública de consulta.	Corregir la configuración o consultar un token válido; no sustituir el token por credenciales.
Swagger no aparece.	La API no está ejecutándose en `Development`.	Revisar `ASPNETCORE_ENVIRONMENT` y la condición de `Program.cs`.	Usar Swagger solo en el entorno de desarrollo configurado; no asumir que está habilitado fuera de él.

La configuración de logging observable establece niveles `Information` para la aplicación y `Warning` para `Microsoft.AspNetCore` en los archivos de configuración. El repositorio no evidencia una plataforma externa de monitoreo, un sistema de trazas centralizado ni archivos de log persistentes; por tanto, este manual no atribuye esas capacidades al sistema.

Los procedimientos específicos de hosting, despliegue productivo, respaldos, restauración, dominio, URL pública y certificados dependerán de la infraestructura seleccionada cuando el Comité y la comunidad autoricen la implementación operativa. Las condiciones técnicas generales ya están documentadas en las secciones siguientes.

> **ESTADO RNF-10**
> Estrategia definida; comprobación operativa condicionada a la futura implementación.

> **ESTADO RNF-11**
> Arquitectura preparada; comprobación operativa condicionada a la futura implementación.

24. CONDICIONES PARA UNA FUTURA IMPLEMENTACIÓN OPERATIVA

La versión desarrollada puede prepararse para alojamiento web. La contratación y configuración de la infraestructura permanente no forman parte de la ejecución actual de la Fase 6. La decisión de adopción operativa y la selección del entorno corresponderán posteriormente al Comité de Agua Potable.

La documentación del sistema conserva las condiciones técnicas necesarias para realizar esa implementación en una etapa posterior, sin afirmar que exista actualmente un entorno productivo. El futuro entorno deberá soportar, como mínimo, los siguientes componentes y capacidades:

- frontend web desarrollado con React y Vite;
- backend desarrollado con ASP.NET Core sobre .NET 8;
- SQL Server compatible con el proveedor utilizado por Entity Framework Core;
- acceso mediante HTTPS;
- configuración segura y externa de secretos;
- almacenamiento y persistencia de la base de datos;
- ejecución controlada de las migraciones de Entity Framework Core;
- mecanismos de respaldo y recuperación.

24.1 Arquitectura de referencia

La arquitectura futura puede representarse conceptualmente de la siguiente manera:

```
Navegador
  |
   HTTPS
  |
```

```
Frontend web
  |
   /api o URL configurada
  |
ASP.NET Core 8
  |
Entity Framework Core
  |
SQL Server
```

Esta arquitectura es conceptual. Puede implementarse mediante distintas infraestructuras compatibles, siempre que conserven la conectividad, la seguridad, la persistencia y los procedimientos de operación requeridos.

24.2 Requisitos del alojamiento

Requisito	Finalidad	Condición de aceptación futura
Ejecución compatible con ASP.NET Core/.NET 8	Ejecutar la API con el framework objetivo del sistema.	La API inicia y atiende solicitudes en un entorno compatible con .NET 8.
Alojamiento del frontend web	Servir la interfaz React/Vite a los usuarios previstos.	El frontend puede cargarse mediante navegador desde el entorno adoptado.
SQL Server compatible	Proporcionar la persistencia relacional requerida por la aplicación.	El backend se conecta a una base SQL Server compatible y puede operar con ella.
HTTPS	Proteger el tránsito entre navegador, frontend y backend.	El acceso previsto se realiza mediante HTTPS con una configuración válida.
Configuración externa de secretos	Evitar que claves, contraseñas y cadenas sensibles formen parte del repositorio.	Los secretos se suministran mediante la configuración segura del entorno y no se exponen en el código ni en registros.
Conectividad frontend/backend	Permitir que la interfaz invoque la API mediante `/api` o una URL configurada.	Las solicitudes principales del frontend alcanzan el backend sin depender de la configuración local de Vite.
Persistencia de datos	Conservar la información administrativa entre ejecuciones.	Los datos permanecen disponibles después de reinicios controlados del sistema.
Aplicación controlada de migraciones	Mantener el esquema de base de datos coordinado con la versión entregada.	Las migraciones se revisan, autorizan y aplican mediante un procedimiento definido para el entorno.
Mecanismos de backup	Conservar puntos de recuperación de la información.	El entorno permite ejecutar y supervisar la estrategia de respaldos definida en la sección 25.
Mecanismo de restauración	Recuperar la información mediante un procedimiento controlado.	El entorno permite restaurar un punto autorizado y verificar posteriormente la integridad y las funciones esenciales.
Acceso administrativo controlado	Limitar la operación de infraestructura y datos a personal autorizado.	Las funciones administrativas del entorno se asignan, protegen y revisan conforme a una autorización definida.

No se establecen cantidades mínimas de CPU, memoria, almacenamiento ni una disponibilidad permanente, porque el proyecto no ha realizado pruebas de capacidad que permitan justificarlas y este documento no define un SLA.

24.3 Condición futura de RNF-11

RNF-11 se considerará comprobado operativamente únicamente cuando se cumplan todas las condiciones siguientes sobre el entorno adoptado:

1. exista un entorno de alojamiento seleccionado y configurado;

2. frontend, backend y persistencia estén disponibles;
3. pueda accederse mediante navegador desde los dispositivos previstos;
4. se comprueben las funciones principales desde ese entorno.

Hasta entonces, el estado de RNF-11 debe describirse como: "Arquitectura preparada; comprobación operativa condicionada a la futura implementación."

25. REQUISITOS DE RESPALDO Y RECUPERACIÓN

El proyecto define como estrategia para la futura infraestructura un respaldo automático diario, la conservación de los últimos 30 respaldos diarios, la posibilidad de generar un respaldo manual autorizado y una restauración controlada. Estos elementos son requisitos para seleccionar y configurar el entorno futuro; no son mecanismos actualmente verificados en producción. Este manual no afirma que se hayan ejecutado respaldos ni restauraciones.

25.1 Respaldo automático

El entorno futuro deberá permitir:

- ejecutar respaldos automáticamente cada día;
- conservar los últimos 30 respaldos diarios;
- supervisar y verificar que cada respaldo se haya realizado;
- proteger el almacenamiento de respaldo frente a accesos no autorizados o exposición pública.

La tecnología concreta para cumplir estas condiciones deberá definirse al seleccionar y configurar la infraestructura. Este manual no impone un producto ni un mecanismo específico.

25.2 Respaldo manual

Deberá existir un procedimiento autorizado para producir un punto de respaldo adicional antes de operaciones sensibles, actualizaciones o tareas de mantenimiento, cuando corresponda. El procedimiento deberá identificar quién puede solicitar o ejecutar la operación, cómo se verifica su resultado y dónde se conserva el punto generado. El repositorio no implementa un botón de backup dentro de la aplicación, por lo que esta capacidad no se atribuye al frontend ni al backend actuales.

25.3 Restauración controlada

La restauración futura deberá seguir un procedimiento conceptual seguro:

1. identificar el respaldo o punto de recuperación que se utilizará;
2. verificar que la restauración esté autorizada;
3. evitar sobrescribir información válida sin un control previo;
4. restaurar mediante el mecanismo previsto por la infraestructura;
5. verificar la integridad de los datos y el acceso al sistema;
6. comprobar las funciones esenciales de la aplicación;
7. documentar la intervención, su resultado y cualquier incidencia.

Los pasos concretos dependerán de la infraestructura finalmente seleccionada. No se incluyen comandos destructivos ni instrucciones específicas de un proveedor inexistente.

25.4 Prueba futura de RNF-10

RNF-10 solo podrá considerarse comprobado operativamente sobre la infraestructura adoptada cuando se cumplan todas las condiciones siguientes:

1. se evidencie la ejecución del respaldo automático;
2. se compruebe la política de retención de los últimos 30 respaldos diarios;
3. se genere un respaldo manual autorizado;
4. se realice una prueba controlada de restauración;
5. se compruebe el funcionamiento posterior del sistema.

Hasta entonces, el estado de RNF-10 debe describirse como: "Estrategia definida; comprobación operativa condicionada a la futura implementación."

25.5 Seguridad de los respaldos

Los respaldos pueden contener información administrativa sensible. Por ello, deben restringirse al personal autorizado y no deben almacenarse públicamente. Las credenciales asociadas a su ejecución, supervisión o restauración deben mantenerse fuera del repositorio y suministrarse mediante mecanismos seguros de configuración del entorno.

26. ALTERNATIVAS DE ALOJAMIENTO PARA UNA FUTURA IMPLEMENTACIÓN

Esta sección tiene carácter informativo. La arquitectura del sistema no depende de un proveedor comercial específico. La futura infraestructura deberá seleccionarse de acuerdo con los requisitos descritos en las secciones 24 y 25.

La información de esta sección no selecciona un proveedor, no constituye una cotización, no implica que se haya contratado infraestructura y no implica que el sistema esté en producción. Tampoco modifica el estado de RNF-10 ni RNF-11, que permanecen condicionados a la futura implementación y comprobación operativa.

> **INFORMACIÓN FECHADA**
> Los precios, planes y características mencionados en esta sección corresponden a información pública consultada en octubre de 2026 y pueden cambiar. Antes de cualquier contratación deberán verificarse nuevamente en los sitios oficiales de los proveedores.

26.1 Alternativa A: SmarterASP.NET

SmarterASP.NET se documenta como una alternativa económica de hosting administrado para una futura implementación compatible con la arquitectura del sistema. La información pública consultada en octubre de 2026 muestra los siguientes planes y capacidades:

- ASP.NET Basic: desde US$2.95/mes;
- ASP.NET Advance: desde US$4.95/mes;

- ASP.NET Premium: desde US$7.95/mes;
- soporte publicado para .NET 8.x;
- soporte para SQL Server;
- SSL disponible;
- backup y restore de MSSQL disponibles;
- conexión remota a SQL Server disponible.

También se publica una prueba gratuita de 60 días sin tarjeta. Esta prueba se considera únicamente una modalidad de evaluación y no constituye una solución productiva permanente.

26.2 Backups de SmarterASP.NET

Según la información pública consultada, los backups de base de datos se realizan por defecto cada dos días. También se publica un complemento CustomBackup para programar respaldos adicionales; los archivos generados mediante ese mecanismo se conservan durante una semana. El panel permite crear y descargar respaldos manuales, y existe un procedimiento para restaurar archivos de respaldo MSSQL.

Estas características no permiten afirmar por sí solas que SmarterASP.NET satisface RNF-10. La política publicada no coincide directamente con la estrategia del proyecto de respaldo automático diario y conservación de los últimos 30 respaldos diarios. Una futura contratación requeriría comprobar y configurar un mecanismo adicional que permita cumplir exactamente esa política.

26.3 Alternativa B: Microsoft Azure

Microsoft Azure puede implementar conceptualmente la arquitectura mediante Azure App Service para la aplicación web y el backend, y Azure SQL Database para la persistencia.

No se establece un precio mensual fijo para la solución completa. Los precios de Azure son variables según la región, el nivel o SKU, los recursos, la modalidad de compra y el consumo o la configuración adoptada.

La documentación oficial contempla App Service F1 gratuito para experimentación y aprendizaje. Esta modalidad no está soportada para cargas productivas y no dispone de SLA, por lo que no debe presentarse como una solución productiva permanente.

26.4 Backups de Azure SQL

Azure SQL realiza backups automáticos y permite recuperación a un punto en el tiempo. En los niveles compatibles, la retención de corto plazo puede configurarse entre 1 y 35 días. Azure SQL Basic está limitado a una retención de 1 a 7 días.

Por ello, una futura implementación que deba satisfacer los 30 días definidos por RNF-10 tendría que seleccionar una configuración compatible con al menos 30 días de retención. La disponibilidad de backups automáticos o de recuperación a un punto en el tiempo no implica, sin configuración y prueba del entorno adoptado, que RNF-10 esté verificado.

26.5 Portabilidad y archivado mediante BACPAC en Azure SQL

Azure SQL permite exportar el esquema y los datos mediante un archivo BACPAC, que puede almacenarse y posteriormente importarse para fines de archivado o portabilidad entre entornos compatibles. Microsoft indica que los archivos BACPAC no están destinados a utilizarse como mecanismo de backup y restore.

Por ello, un archivo BACPAC no sustituye la estrategia de backup automático, retención, supervisión, respaldo manual autorizado y restauración controlada definida en la sección 25.

26.6 Tabla comparativa

Alternativa	Compatibilidad tecnológica	Costo de referencia	Backup/restauración	Consideración para RNF-10
SmarterASP.NET	.NET 8 + SQL Server	Desde US$2.95/mes	Backup y restore disponibles	Requiere verificar y complementar la política para cumplir 30 respaldos diarios.
Microsoft Azure	App Service + Azure SQL	Costo variable según configuración	Backup automático y recuperación a punto en el tiempo	Requiere seleccionar una configuración que permita al menos 30 días de retención; BACPAC se considera para archivado o portabilidad, no como backup.

La tabla es una referencia técnica y económica fechada; no establece una selección, preferencia ni contratación.

26.7 Criterios para una decisión futura

Cuando el Comité decida implementar el sistema, deberá evaluar como mínimo:

- costo sostenible;
- compatibilidad con .NET 8;
- SQL Server;
- HTTPS;
- administración de secretos;
- facilidad de mantenimiento;
- mecanismo de migraciones;
- política de backup;
- retención mínima requerida;
- restauración;
- acceso administrativo;
- soporte técnico.

La decisión deberá basarse en las condiciones vigentes en ese momento y en la verificación actualizada de las capacidades, precios, restricciones y políticas de cada alternativa.

26.8 Fuentes de referencia consultadas

Las siguientes referencias se registran como fuentes textuales de la información utilizada en esta sección, consultadas en octubre de 2026. No se inventan fechas de publicación cuando la fuente no las proporciona:

- Microsoft Azure — App Service pricing.
- Microsoft Learn — Automatic, geo-redundant backups, Azure SQL Database.
- Microsoft Learn — Export a database to a BACPAC file.
- Microsoft Azure — Azure SQL Database pricing.
- SmarterASP.NET — ASP.NET Core Hosting.
- SmarterASP.NET — ASP.NET Hosting Plans.
- SmarterASP.NET — How can I create a custom database backup?
- SmarterASP.NET — How to create and download a database backup.
- SmarterASP.NET — How can I restore MSSQL database to your server?